Fullstack Home Assignment – Mini Food Ordering System

Build a **small fullstack food ordering application** with:

- **Backend:** NestJS + TypeScript
- **Frontend:** Next.js + React + TypeScript
- **Database:** Any DB + ORM of your choice
- **Auth:** JWT

The goal is to demonstrate clean architecture, basic CRUD, authentication, and simple UI integration.

Backend Requirements:

- NestJS (TypeScript)
- ORM (Prisma, TypeORM, Sequelize, etc.)
- JWT authentication
- Minimal Jest coverage (1–2 unit tests)
- Swagger documentation
- Dockerfile for backend

Endpoints

- `POST /auth/register` – Register a customer (name, email, password).
- `POST /auth/login` – Return JWT on successful login.
- `GET /auth/who-am-i` – Return currently authenticated user.
- `GET /restaurants` – List all restaurants.
- `GET /restaurants/:id` – Return restaurant details including its menu.
- `POST /orders` – Create an order for logged-in user.
- `GET /orders/:id` – Return details of a specific order for the logged-in customer.

Frontend Requirements

- Next.js (App or Pages Router)
- React + TypeScript
- Any styling (Tailwind / CSS modules / etc.)
- Fetch/Axios for API calls

Pages

- **/login** – email + password; save JWT; redirect to `/restaurants`.
- **/register** – name, email, password; redirect to login or auto-login.
- **/restaurants** – fetch and list restaurants.
- **/restaurants/[id]**
    - Show restaurant details + menu.
    - User can add menu items to a simple cart (local state is enough).
    - “Place Order” button → send `POST /orders` → redirect to order page.
- **/orders/[id]** – show order details and status.

Deliverables

- Git repository containing:
    - Backend code (NestJS)
    - Frontend code (Next.js)
    - SQL/migrations for DB setup
    - Dockerfile(s)
    - **README** with:
        - How to run backend & frontend
        - How to seed/init DB
        - API base URL and Swagger URL

Scope Notes

- Keep functionality minimal and focused.
- No restaurant admin panel.
- No order status updates except returning a static or initial status.
- No real-time communication required.

Bonus task:

- Real-time order status updates
    -